NYCU Pattern Recognition, Homework 1

311553059 孫郁茹

Part. 1, Coding (60%):

(10%) Linear Regression Model - Closed-form Solution

1. (10%) Show the weights and intercepts of your linear model.

```
2024-04-03 03:55:26.371 | INFO     | __main__:main:97 - LR_CF.weights=array([2.8491
883  , 1.0188675 , 0.48562739, 0.1937254 ]), LR_CF.intercept=-33.8223
```

(40%) Linear Regression Model - Gradient Descent Solution

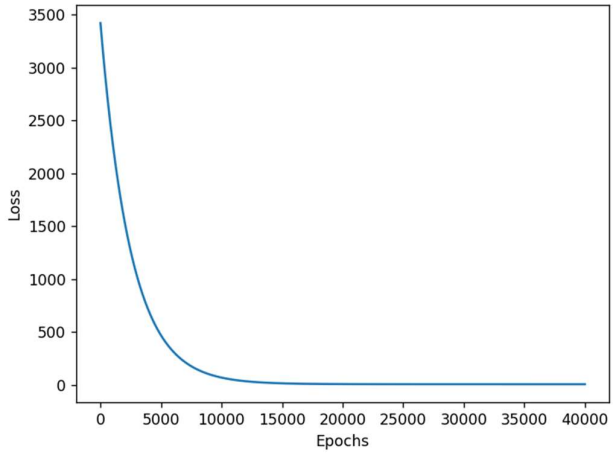
2. (0%) Show the learning rate and epoch (and batch size if you implement mini-batch gradient descent) you choose.

```
LR_GD.fit(train_x, train_y, learning_rate=1e-4, epochs=40000)
```

3. (10%) Show the weights and intercepts of your linear model.

```
2024-04-03 03:57:54.201 | INFO     | __main__:main:102 - LR_GD.weights=array([ 7.40
164001, 17.64529158,  0.82114212,  0.55682016]), LR_GD.intercept=55.1990
```

4. (10%) Plot the learning curve. (x-axis=epoch, y-axis=training loss)



5. (20%) Show your error rate between your closed-form solution and the gradient descent solution.

```
2024-04-03 03:59:28.153 | INFO     | __main__:main:111 - Prediction difference: 37.
0059
2024-04-03 03:59:28.154 | INFO     | __main__:main:116 - mse_cf=4.1997, mse_gd=4.20
12. Difference: 0.034%
```

6. (Bonus 5 points) Implement the L1 regularization into the gradient descent method and show the weights, intercept, and learning curve.

(10%) Code Check and Verification

7. (10%) Lint the code and show the PyTest results.

```
C:\Users\sunruru\Downloads\release\release>python -m flake8

C:\Users\sunruru\Downloads\release\release>python -m pytest ./test_main.py -s
============================= test session starts ==============================
platform win32 -- Python 3.10.11, pytest-8.1.1, pluggy-1.4.0
rootdir: C:\Users\sunruru\Downloads\release\release
collected 2 items

test_main.py 2024-04-03 04:00:15.821 | INFO     | test_main:test_regression_cf:27 -
 model.weights=array([[3.]]), model.intercept=array([4.])
.F

=================================== FAILURES ===================================
_______________________________ test_regression_gd _______________________________

sample_data = (array([[-100.        ],
       [ -97.97979798],
       [ -95.95959596],
       [ -93.93939394],
       [ -91.91919192...     [ 279.75757576],
       [ 285.81818182],
       [ 291.87878788],
       [ 297.93939394],
       [ 304.        ]])),
slope_and_intercept = (3, 4)

    def test_regression_gd(sample_data, slope_and_intercept):
        x, y = sample_data
        model = LinearRegressionGradientdescent()
>       model.fit(x, y, learning_rate=1e-4, epochs=70000)

test_main.py:37:
- - - - - - - - - - - - - - - - - - - - - - - - - - - - - - - - - - - - - - - -

self = <main.LinearRegressionGradientdescent object at 0x000001A4516E8E20>
X = array([[-1.71481604],
       [-1.68017329],
       [-1.64553055],
       [-1.6108878 ],
       [-1.57624505],
       [-1.6108878 ],
       [-1.57624505],
       [...16023 ],
       [ 1.57624505],
       [ 1.6108878 ],
       [ 1.64553055],
       [ 1.68017329],
       [ 1.71481604]])
y = array([[-296.        ],
       [-289.93939394],
       [-283.87878788],
       [-277.81818182],
       [-271.75757576]...     [ 279.75757576],
       [ 285.81818182],
       [ 291.87878788],
       [ 297.93939394],
       [ 304.        ]])
learning_rate = 0.0001, epochs = 70000

    def fit(self, X, y, learning_rate: float = 0.001, epochs: int = 1000):
        # X = scale_features(X)
        X = self.scale_features(X, is_training=True)
        n_samples, n_features = X.shape
        self.weights = np.zeros(n_features)
        self.intercept = 0
        losses = []

        for _ in range(epochs):
            y_pred = self.predict(X, is_training=True)
            error = y_pred - y

            gradient = (2 / n_samples) * X.T @ error
            intercept_gradient = (2 / n_samples) * np.sum(error)

>           self.weights -= learning_rate * gradient
E           ValueError: non-broadcastable output operand with shape (1,) doesn't match the broadcast shape (1,100)

main.py:61: ValueError
=========================== short test summary info ============================
FAILED test_main.py::test_regression_gd - ValueError: non-broadcastable output operand with shape (1,) doesn't match the...
========================= 1 failed, 1 passed in 0.94s ==========================
```

Part. 2, Questions (40%):

1. (10%) Please describe the Vanishing Gradient Problem in detail, and provide **at least two s olutions** to overcome this problem.

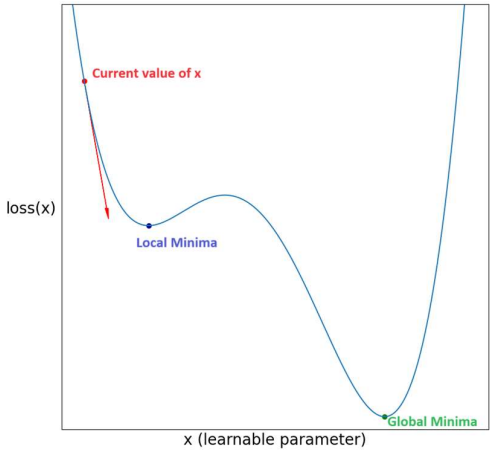
   [Your answer here.]

   It is commonly encountered during the training of deep neural networks. When the gradien t of the loss function with respect to the model parameters become extremely small, learni ng becomes slow or stalled. The gradient magnitude typically is expected to decrease as th e sequence length increases.

   Solutions:

   1) Batch normalization
   2) Other activation functions. For example, ReLU since it only saturates in one direction.

2. (15%) Gradient descent often suffers from the issue of getting stuck at local minima (refer to the figure provided). Please provide **at least two methods** to overcome this problem and discuss how these methods work.



   [Your answer here.]

   1) Random initialization. Start traing from different initial values and train the model mul tiple times independently. Then, we may find a better global min, not a local min.
   2) Stochastic Gradient Descent (SGD). Instead of using the entire dataset, SGD randomly sampling a subset of the dataset at each iteration. The randomness and variability help escape local min.

3. (15%) What are the basic assumptions of Linear regression between the features and the ta rget? How can techniques help Linear Regression extend beyond these assumptions? Pleas e at least answer one technique.

   [Your answer here.]

1) linearity: the target variable can be expressed as a linear combination of the features
2) the error term must be normally distributed
3) the error term must have constant variance
4) the independent variables should not be correlated
5) no correlation between the error terms

Polynomial regression: allow the relationship between the independent variables and the d ependent variable to be an n-th degree polynomial. (nonlinear relationship)